



K.RAMAKRISHNAN
COLLEGE OF ENGINEERING
An Autonomous Institution

Permanently Affiliated to Anna University Chennai, Approved by AICTE New Delhi,
ISO 9001:2015, 14001:2015 certified institution, Accredited by NBA and with A grade by NAAC
Samayapuram, Tiruchirappalli – 621 112, Tamilnadu, India.



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CGB1231 – Artificial Intelligence and Machine Laboratory

Course Code	Course Title	Semester	Category	Periods/Week				C
				L	T	P	S	
CGB1231	Artificial Intelligence and Machine Laboratory	IV	PC	0	0	2	0	1
Pre-requisite	Python	Course Type	Practical	Learning Level				BTL4
Course Objectives:								
To impart knowledge on Artificial intelligence concepts using Machine Learning Algorithms								
Course Outcomes:								
Upon completion of the course, the students will be able to:								
Course Outcomes								
CO1	Apply the basic principles of problem solving using AI techniques.							
CO2	Calibrate the knowledge representation using AI techniques							
CO3	Construct the Learning Models and evaluate the results.							
CO4	Apply Classification algorithms and generate the results.							
CO5	Apply Clustering algorithms to appropriate problems and generate the solutions.							

Syllabus	
List of Experiments / Exercises	
1	Implementation of Uninformed search algorithms: - Construct Breadth-First Search(BFS) algorithm to traverse a graph in a systematic way using python - Construct Depth-First Search(DFS) and algorithm to traverse a tree data structure in a systematic way using python
2	Implementation of N-Queen Problem The task is to place N queens on an N×N chessboard in such a way that none of the queens is under attack. Implement this N Queen problem using Python.
3	Implementation of Propositional model Construct and implement propositional model checking algorithms using python
4	Implementation of Chatbot Model To create a chatbot that can assist university students with their common queries related to campus resources, course information, schedules, and general student life.
5	Implementation of Naïve Bayes and Bayesian Network - Construct and implement Naïve Bayes using Naïve classifier methods - Implement Bayesian Network using Python
6	Implementation of Regression Model and Logistic Regression - Construct and Implement python program for regression model using linear regression. - Construct and Implement python program for regression model using logistic regression.
7	Implementation of Decision Tree and Random Forest Tree. - Construct python program Decision tree using Gaussian classifier. - Construct python program for random forest tree.
Total Hours : 30	

EXPT NO : 1(a) Implementation of Uninformed search algorithms -Breadth-First Search(BFS)
DATE

Aim:

To implements the simple uniformed search algorithm breadth first search methods using python.

Procedure:

1. Start by putting any one of the graph's vertices at the back of the queue.
2. Now take the front item of the queue and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add those which are not within the visited list to the rear of the queue.
4. Keep continuing steps two and three till the queue is empty.

Program:

```
from collections import deque

class Graph:
    def __init__(self, directed=True):
        self.edges = {}
        self.directed = directed
    def add_edge(self, node1, node2, reversed=False):
        try:
            neighbors = self.edges[node1]
        except KeyError:
            neighbors = []
        if node2 not in neighbors: # Avoid duplicates
            neighbors.append(node2)
        self.edges[node1] = neighbors
        if not self.directed and not reversed:
            self.add_edge(node2, node1, True)
    def neighbors(self, node):
        try:
            return self.edges[node]
        except KeyError:
            return []
    def breadth_first_search(self, start, goal):
```

```

found = False
fringe = deque([start])
visited = set([start])
came_from = {start: None}
print('{:11s} | {}'.format('Expand Node', 'Fringe'))
print(' ')
print('{:11s} | {}'.format('-', start))
while not found and len(fringe):
    current = fringe.pop()
    print('{:11s}'.format(current), end=' | ')
    if current == goal:
        found = True
        break
    for node in self.neighbors(current):
        if node not in visited:
            visited.add(node)
            fringe.appendleft(node)
            came_from[node] = current
    print(' ', '.join(fringe))
    if found:
        print()
        return came_from
    else:
        print('No path from {} to {}'.format(start, goal))
        return None
    @staticmethod
    def print_path(came_from, goal):
        parent = came_from[goal]
        if parent:
            Graph.print_path(came_from, parent)
        else:
            print(goal, end="")
        return
    print(' =>', goal, end="")
    def __str__(self):
        return str(self.edges)
    # Create the graph

```

```

graph = Graph(directed=False)
graph.add_edge('A', 'B')
graph.add_edge('A', 'S')
graph.add_edge('S', 'G')
graph.add_edge('S', 'C')
graph.add_edge('C', 'F')
graph.add_edge('G', 'F')
graph.add_edge('C', 'D')
graph.add_edge('C', 'E')
graph.add_edge('E', 'H')
graph.add_edge('G', 'H')
# Perform BFS
start, goal = 'A', 'H'
traced_path = graph.breadth_first_search(start, goal)
# Print the path if found
if traced_path:
    print('Path:', end=' ')
    Graph.print_path(traced_path, goal)
    print()

```

Output:

Expand Node | Fringe

```

-      | A
A      | S, B
B      | S
S      | C, G
G      | H, F, C
C      | E, D, H, F
F      | E, D, H
H      |
Path: A => S => G => H

```

Result:

Thus, the program for breadth first search was executed and outputs were verified. using a python program.

EXPT NO : 1(b) Implementation of Uninformed search algorithms -Depth-First Search(DFS)
DATE

AIM:

To implement and understand the Depth First Search (DFS) algorithm for traversing or searching a graph/tree structure in Artificial Intelligence (AI).

ALGORITHM:

1. **Initialize the Graph**
 - Represent the graph using an adjacency list or adjacency matrix.
 - Mark all nodes as unvisited.
2. **Start DFS from a Given Node**
 - Choose a starting node (source).
 - Push the starting node onto a stack.
 - Mark the node as visited.
3. **Explore Nodes**
 - While the stack is not empty:
 - Pop a node from the stack.
 - Process the node (print or store the result).
 - Push all its **unvisited** adjacent nodes onto the stack.
 - Mark the newly visited nodes to avoid revisits.
4. **Repeat Until Stack is Empty**
 - Continue exploring nodes until all reachable nodes have been visited.
5. **End of Algorithm**
 - The traversal is complete once all nodes are visited.
 - If searching for a specific goal, stop when the goal node is found.

PROGRAM:

```
class Graph:
    def __init__(self):
        self.edges = {}

    def add_edge(self, node1, node2):
        if node1 not in self.edges:
            self.edges[node1] = []
        if node2 not in self.edges:
            self.edges[node2] = []
        self.edges[node1].append(node2)
        self.edges[node2].append(node1) # Since it's an undirected graph

    def dfs(self, start, goal, visited=None):
        if visited is None:
            visited = set()

        print(start, '=>', end=' ')
        if start == goal:
            return True

        visited.add(start)
        for neighbor in self.edges.get(start, []):
            if neighbor not in visited:
                if self.dfs(neighbor, goal, visited):
                    return True
```

```
return False
```

```
# Create the graph
graph = Graph()
graph.add_edge('A', 'B')
graph.add_edge('A', 'C')
graph.add_edge('B', 'D')
graph.add_edge('C', 'E')
graph.add_edge('D', 'F')
graph.add_edge('E', 'F')

dfs_start, dfs_goal = 'A', 'F'
print("DFS Path:")
graph.dfs(dfs_start, dfs_goal)
```

OUTPUT :

DFS Path:

A => B => D => F

RESULT:

Thus the Depth First Search (DFS) algorithm for traversing or searching a graph/tree structure in Artificial Intelligence (AI) was implemented using python code and hence the output has been verified successfully.

EXPT NO :2

8-queens Problem

DATE

AIM:

To Solve 8-Queens Problems using Python language.

ALGORITHM:

- This **pseudocode uses a backtracking algorithm** to find a solution to the 8 Queen problem, which consists of placing 8 queens on a chessboard in such a way that no two queens threaten each other.
- The algorithm starts by placing a queen on the first column, then it proceeds to the next column and places a queen in the **first safe** row of that column.
- If the algorithm reaches the 8th column and all **queens are placed in a safe position**, it prints the board and returns true.
If the algorithm is unable to place a queen in a safe position in a certain column, it backtracks to the previous column and tries a different row.
- The "isSafe" function **checks** if it is safe to place a queen on a certain row and column by checking if there are any queens in the same row, diagonal or anti-diagonal.
- It's worth to notice that this is just a high-level **pseudocode** and it might need to be adapted depending on the specific implementation and language you are using.

PROGRAM:

```
# Ask for the number of queens
```

```
N = int(input("Enter the number of queens: "))
```

```
print(f"You entered: {N}")
```

```
# Create a chessboard (NxN matrix) with all elements set to 0
```

```
board = [[0] * N for _ in range(N)]
```

```
def attack(i, j):
```

```
    # Checking vertically and horizontally
```

```
    for k in range(N):
```

```
        if board[i][k] == 1 or board[k][j] == 1:
```

```
            return True
```

```
    # Checking diagonally
```

```
    for k in range(N):
```

```
        for l in range(N):
```



```

        if (k + l == i + j) or (k - l == i - j):
            if board[k][l] == 1:
                return True
    return False

def N_queens(n):
    if n == 0:
        return True

    for i in range(N):
        for j in range(N):
            if not attack(i, j) and board[i][j] != 1:
                board[i][j] = 1
                if N_queens(n - 1):
                    return True
                board[i][j] = 0 # Backtracking

    return False

# Solve N-Queens
N_queens(N)

# Print the board
for row in board:
    print(row)

```

OUTPUT:

Enter the number of queens: 5

You entered: 5

[1, 0, 0, 0, 0]

[0, 0, 1, 0, 0]

[0, 0, 0, 0, 1]

[0, 1, 0, 0, 0]

[0, 0, 0, 1, 0]

RESULT:

Thus the eight queens problem solved using python code and hence the output has been verified successfully.

EXPT NO : 5(a) Implementation of NaïveBayes using Navie classifier methods

AIM:

To construct and implement Navie bayes using Navie classifier methods using Python.

Algorithm

1. The code starts by loading the iris dataset.
2. The data is then split into a training and test set of equal size.
3. Next, a Gaussian Naive Bayes classifier is trained using the training set.
4. Then predictions are made on the test set with accuracy scores calculated for each prediction.
5. Finally, a confusion matrix is created to show how well each prediction was classified as correct or incorrect
6. The code is used to train a Gaussian Naive Bayes classifier and then use it to make predictions.
7. The code prints the model's predictions, as well as the test set's output for comparison.

Program:

```
from sklearn import datasets

from sklearn.metrics import confusion_matrix, accuracy_score

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

# Load the Iris dataset

iris = datasets.load_iris()
```

```
X = iris.data
```

```
Y = iris.target
```

```
# Split the dataset into training and testing sets
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=1/3, random_state=42)
```

```
# Train a Gaussian Naive Bayes classifier
```

```
model = GaussianNB()
```

```
model.fit(X_train, Y_train)
```

```
# Make predictions on the test set
```

```
model_predictions = model.predict(X_test)
```

```
# Print predictions and actual labels
```

```
print("\nPredictions:", model_predictions)
```

```
print("\nActual labels:", Y_test)
```

```
# Calculate and print the accuracy score
```

```
accuracyScore = accuracy_score(Y_test, model_predictions)
```

```
print("\nAccuracy Score:", accuracyScore)
```

```
# Create and print the confusion matrix
```

```
cm = confusion_matrix(Y_test, model_predictions)
```

```
print("\nConfusion Matrix:\n", cm)
```

Output:

```
Predictions: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 2 2 1 1 2 0 2 0 2 2 2 2 2 0
0 0 0 1 0 0 2 1
0 0 0 2 1 1 0 0 1 1 2 1 2]
```

```
Actual labels: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2
0 0 0 0 1 0 0 2 1
0 0 0 2 1 1 0 0 1 2 2 1 2]
```

Accuracy Score: 0.96

Confusion Matrix:

```
[[19  0  0]
 [ 0 14  1]
 [ 0  1 15]]
```

Result

Thus the program to implement Naïve Bayes Model is implemented and executed successfully

EXPT NO : 5(b) Implementation of Bayesian Network using Python

Aim

To construct and implement Bayesian Network using Python.

Algorithm

1. Open command prompt and type, pip install torch torchvision torchaudio. !pip install torchbnn in a command prompt in python idle.
2. Import Libraries: Import necessary libraries, including torch, torchbnn, datasets from sklearn, and matplotlib.
3. Load Dataset: Load the Iris dataset using datasets.load_iris() and prepare the data and target as tensors using torch.from_numpy().
4. Create Model: Define a Bayesian Neural Network model using bnn.BayesLinear layers and nn.ReLU() activation function.
5. Define Loss Functions: Set up the cross-entropy loss function and Kullback-Leibler (KL) divergence loss using nn.CrossEntropyLoss() and bnn.BKLLoss().
6. Initialize Optimizer: Use the Adam optimizer with a learning rate of 0.01 to update the model parameters.
7. Training Loop: Run a loop for 3000 steps, where for each step, the model is fed the data tensor, and the losses (cross-entropy + KL divergence) are calculated.
8. Backward Pass: Backpropagate the total loss and update the model parameters using optimizer.step().
9. Compute Accuracy: After the loop ends, compute the model's accuracy by comparing predicted values with the actual target values.
10. Output Final Results: Print the final accuracy, cross-entropy loss, and KL divergence after training.
11. Plot Results: Define a function to plot two subplots, one showing the true labels and the other showing the predicted labels for visual comparison.

Program:

```
import numpy as np
```

```
from sklearn import datasets

import torch

import torch.nn as nn

import torch.optim as optim

import torchbnn as bnn

import matplotlib.pyplot as plt


# Load Iris dataset

dataset = datasets.load_iris()

data = dataset.data

target = dataset.target


# Convert dataset to PyTorch tensors

data_tensor = torch.from_numpy(data).float()

target_tensor = torch.from_numpy(target).long()


# Define the Bayesian Neural Network model

model = nn.Sequential(

    bnn.BayesLinear(prior_mu=0, prior_sigma=0.1, in_features=4, out_features=100),

    nn.ReLU(),

    bnn.BayesLinear(prior_mu=0, prior_sigma=0.1, in_features=100, out_features=3)

)


# Define the loss functions

cross_entropy_loss = nn.CrossEntropyLoss()
```



```
klloss = bnn.BKLLoss(reduction='mean', last_layer_only=False)
```

```
# Define the optimizer
```

```
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

```
# Training loop
```

```
for step in range(3000):
```

```
    # Forward pass
```

```
    models = model(data_tensor)
```

```
    # Calculate Cross Entropy and KL loss
```

```
    cross_entropy = cross_entropy_loss(models, target_tensor)
```

```
    kl = klloss(model)
```

```
    # Total cost
```

```
    total_cost = cross_entropy + 0.01 * kl
```

```
    # Backward pass
```

```
    optimizer.zero_grad()
```

```
    total_cost.backward()
```

```
    optimizer.step()
```

```
# Final prediction and accuracy calculation
```

```
models = model(data_tensor)
```

```
_, predicted = torch.max(models.data, 1)
```

```

correct = (predicted == target_tensor).sum()

accuracy = 100 * float(correct) / target_tensor.size(0)


# Final output

cross_entropy = cross_entropy_loss(models, target_tensor)

kl = klloss(model)


print(f'Final Accuracy: {accuracy:.2f}%')

print(f'Final CE: {cross_entropy.item():.2f}, Final KL: {kl.item():.2f}')


# Function to plot the results

def draw_graph(predicted):

    fig = plt.figure(figsize=(16, 8))

    fig_1 = fig.add_subplot(1, 2, 1)

    fig_2 = fig.add_subplot(1, 2, 2)


    # Plot the true labels

    z1_plot = fig_1.scatter(data[:, 0], data[:, 1], c=target, marker='v')

    # Plot the predicted labels

    z2_plot = fig_2.scatter(data[:, 0], data[:, 1], c=predicted)


    plt.colorbar(z1_plot, ax=fig_1)

    plt.colorbar(z2_plot, ax=fig_2)


    fig_1.set_title("REAL")

```

```
fig_2.set_title("PREDICT")
```

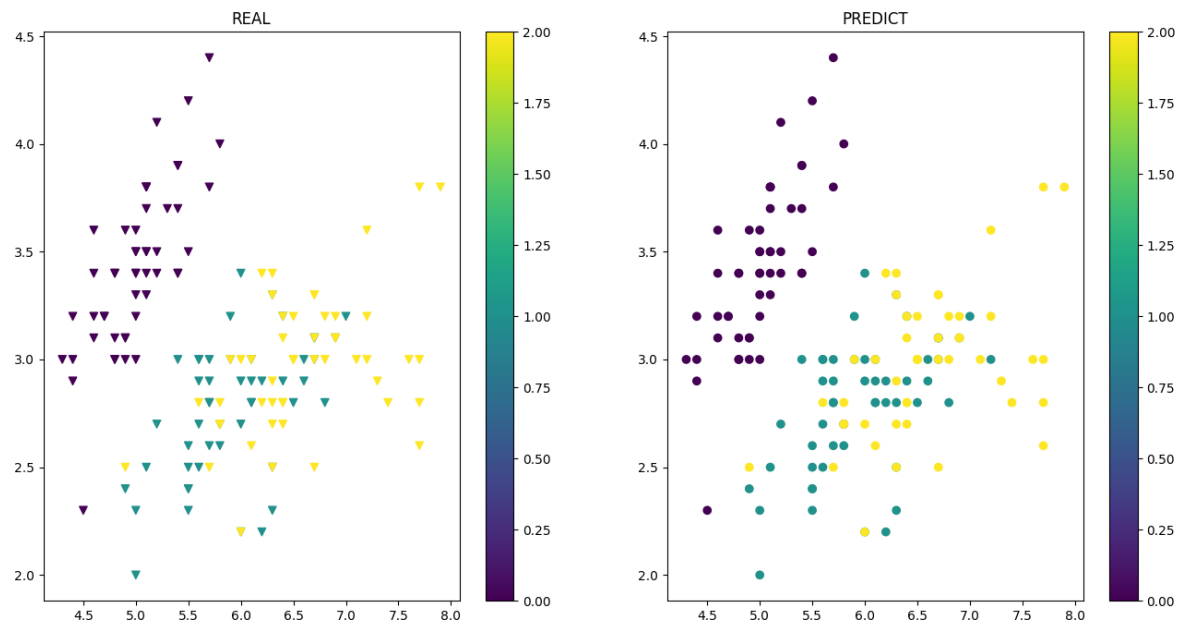
```
plt.show()
```

```
# Final predictions and plot
```

```
draw_graph(predicted)
```

Output:

Final Accuracy: 96.67%
Final CE: 0.06, Final KL: 3.21



Result

Thus, the program to implement Bayesian Networks and perform inferences is implemented and executed successfully

EXPTNO :6(a) Implementation of Regression Model using linear regression

Aim

To Construct and Implement python program for regression model using linear regression.

Algorithm

1. Import Libraries: Import necessary libraries like pandas, numpy, and sklearn.
2. Load and Preprocess Data: Load the dataset, handle missing values (e.g., fill missing values with the median), and split it into features (X) and target (y).
3. Train the Model: Use Linear Regression to train the model on the training data.
4. Make Predictions: Use the trained model to predict prices for new data.
5. Evaluate the Model: Calculate performance metrics like Mean Squared Error (MSE) and R-squared to evaluate the model.

Code:

```
import pandas as pd

from sklearn import linear_model

from sklearn.model_selection import train_test_split

from sklearn.metrics import mean_squared_error, r2_score

# Load the dataset

df = pd.read_csv("C:\\Users\\Renuka\\Desktop\\AI ML LAB\\homeprice.csv")

# Strip any leading or trailing spaces from column names

df.columns = df.columns.str.strip()

# Data Preprocessing: Fill missing values with median value of the column

df['bedrooms'] = df['bedrooms'].fillna(df['bedrooms'].median())

df['Age (years)'] = df['Age (years)'].fillna(df['Age (years)'].median())
```

```
# Features and target

X = df.drop('Price ($)', axis='columns') # Features (Area, Bedrooms, Age)

y = df['Price ($)'] # Target (Price)

# Split the data into training and test sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a linear regression model

reg = linear_model.LinearRegression()

# Train the model

reg.fit(X_train, y_train)

# Make predictions on the test set

y_pred = reg.predict(X_test)

# Model Evaluation: Mean Squared Error and R-squared

mse = mean_squared_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)

print(f'Mean Squared Error: {mse}')

print(f'R-squared: {r2}') # Model coefficients and intercept

print(f'Model Coefficients: {reg.coef_}')

print(f'Model Intercept: {reg.intercept_}')

# Make predictions on new data

new_data = [[3000, 3, 40]] # Example: 3000 sqft, 3 bedrooms, 40 years old

predicted_price = reg.predict(new_data)

print(f'Predicted Price for 3000 sqft, 3 bedrooms, 40 years old: {predicted_price[0]}')

# Another example: 2500 sqft, 4 bedrooms, 5 years old
```

```
new_data_2 = [[2500, 4, 5]]
```

```
predicted_price_2 = reg.predict(new_data_2)
```

```
print(f'Predicted Price for 2500 sqrft, 4 bedrooms, 5 years old: {predicted_price_2[0]}')
```

Original Dataset:

Save it as csv file

Area (sqft)	bedrooms	Age (years)	Price (\$)
2600	3	20	550000
3000	4	15	565000
3200	3	18	610000
3600	3	30	595000
4000	5	8	760000
4200	4	5	810000
2500	2	40	450000
2700	2	35	480000
3300	3	12	650000
3700	4	10	720000
4100	5	7	800000
2200	2	50	400000
2900	3	25	520000
3100	3	20	580000
3800	4	10	710000

Output:

Mean Squared Error: 1847518337.3775837

R-squared: 0.8918877435864873

Model Coefficients: [152.24076853 -5591.98289037 -3788.21617609]

Model Intercept: 205091.07412720926

Predicted Price for 3000 sqrft, 3 bedrooms, 40 years old:
493508.78400151443

Predicted Price for 2500 sqrft, 4 bedrooms, 5 years old: 544383.9830094788

Result:

Thus, the python program for linear regression model was executed successfully.

EXPTNO :6(b) Implementation of Regression Model using logisticregression

Construct and Implement python program for regression model using logistic regression.

AIM:

To Construct and Implement python program for regression model using logisticregression.

Algorithm

Step 1: Data Pre Processing

- Import necessary libraries.
- Import the dataset.

Step 2:Extracting Independent and dependent Variable

Identify the independent variables (features) and dependent variable (target).

Step3:Splitting the dataset into training and test set.

Use train_test_split to divide the data into training and test sets.

Step4:feature Scaling

Step5:Fitting Logistic Regression to the training set

Step6:Predicting the test set result

Step 2:Extracting Independent and dependent Variable

Step3:Splitting the dataset into training and test set.

Step4:feature Scaling

Step5:Fitting Logistic Regression to the training set

Step6:Predicting the test set result

Code:

```
import pandas as pd

from matplotlib import pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

import math
```


Step 1: Importing the dataset

try:

```
df = pd.read_csv("C:\\Users\\Renuka\\Desktop\\AI ML LAB\\insurance.csv") # Replace with  
your actual dataset file path
```

```
print("Dataset preview:")
```

```
print(df.head()) # Print first few rows to ensure the data is loaded
```

except FileNotFoundError:

```
print("Error: The file 'insurance_data.csv' was not found. Please check the file path.")
```

```
exit()
```

Step 2: Visualizing the data

```
plt.scatter(df['Age'], df['Bought insurance'], marker='+', color='red')
```

```
plt.xlabel('Age')
```

```
plt.ylabel('Bought Insurance (0 = No, 1 = Yes)')
```

```
plt.title('Age vs Bought Insurance')
```

```
plt.show()
```

Step 3: Checking for necessary columns

if 'Age' not in df.columns or 'Bought insurance' not in df.columns:

```
print("Error: The dataset does not contain required columns 'Age' and 'Bought insurance'.")
```

```
exit()
```

Step 4: Splitting the dataset into training and test sets

```
X_train, X_test, y_train, y_test = train_test_split(df[['Age']], df['Bought insurance'],  
train_size=0.8)
```

```
print("Test data preview:")
```

```
print(X_test.head()) # Ensure we have test data
```

Step 5: Creating the Logistic Regression model

```
model = LogisticRegression()
```

```
model.fit(X_train, y_train)
```

```

# Step 6: Predictions and evaluation

y_predicted = model.predict(X_test)

print("\nPredicted values for test data:")

print(y_predicted)

# Model accuracy score

accuracy = model.score(X_test, y_test)

print(f"\nModel Accuracy: {accuracy:.2f}")

# Step 7: Predicted probabilities

print("\nPredicted probabilities for the test data:")

print(model.predict_proba(X_test)) # Show probabilities for test data

# Step 8: Coefficients of the model

print("\nModel coefficient (m):", model.coef_)

print("Model intercept (b):", model.intercept_)

# Step 9: Sigmoid function and manual predictions

def sigmoid(x):

    """Sigmoid function for logistic regression prediction"""

    return 1 / (1 + math.exp(-x))

def prediction_function(age):

    """Function to predict probability for a given age"""

    # Calculate the log-odds (z) for the logistic regression

    z = model.coef_[0][0] * age + model.intercept_[0]

    # Apply the sigmoid function to get the probability

    y = sigmoid(z)

    return y

# Step 10: Making manual predictions for certain ages

age = 35

```

```

pred_35 = prediction_function(age)

print(f"\nPredicted probability for age {age}: {pred_35:.4f}")

age = 43

pred_43 = prediction_function(age)

print(f"Predicted probability for age {age}: {pred_43:.4f}")

```

Dataset

Save it as csv

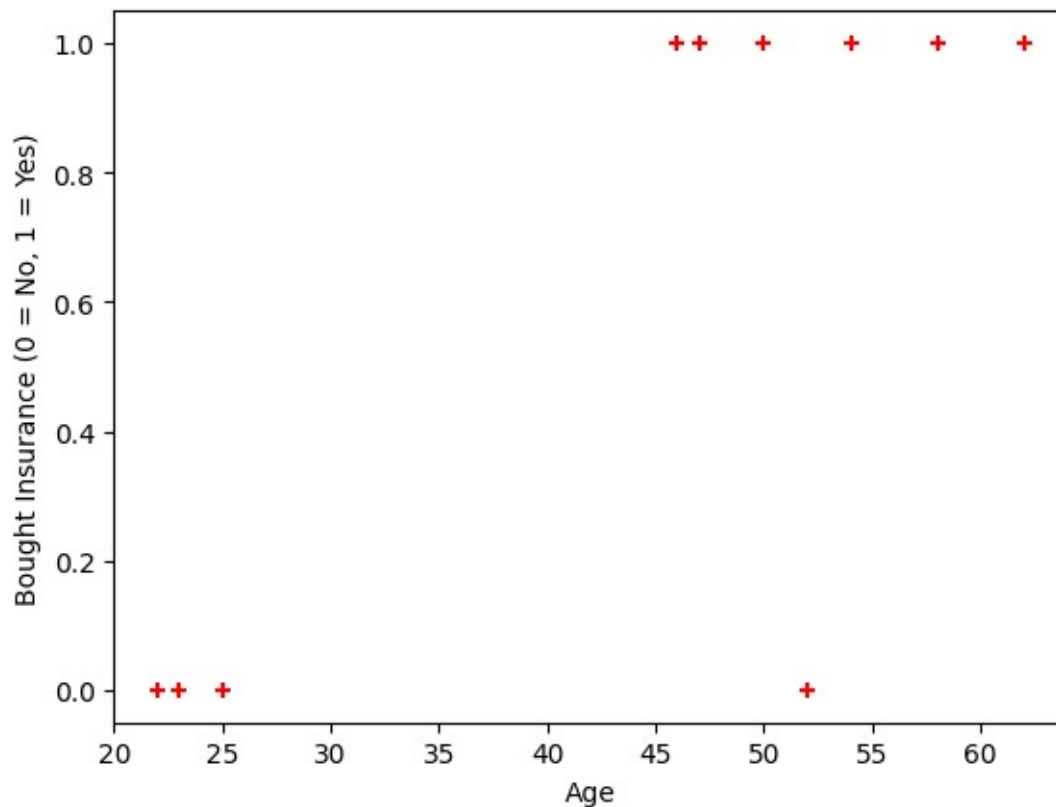
Age	Bought insurance
22	0
25	0
47	1
52	0
46	1
62	1
23	0
58	1
50	1
54	1

Output:

```

Dataset preview:
Age  Bought insurance
0    22                0
1    25                0
2    47                1
3    52                0
4    46                1

```



Test data preview:

```
Age
5   62
8   50
```

Predicted values for test data:

```
[1 1]
```

Model Accuracy: 1.00

Predicted probabilities for the test data:

```
[[0.04212471 0.95787529]
 [0.23893469 0.76106531]]
```

Model coefficient (m): [[0.16379618]]

Model intercept (b): [-7.03128012]

Predicted probability for age 35: 0.21443209738034713

Predicted probability for age 43: 0.5029888835347912

Result:

Thus, the python program for logistic regression model was executed successfully.

EXPTNO :7 (a) **Implementation of Decision Tree**

Construct python program Decision tree using Gaussian mixture model.

Aim:

To Construct python program Decision tree using Gaussian mixture model and visualize graph .

Procedure:

1. Required libraries, namely pandas, numpy, matplotlib, and scikit-learn with the modules for preprocessing, clustering, classification and evaluation, are imported.
2. The dataset is loaded into the local directory.
3. For the data preprocessing, the label encoder is performed.
4. For feature scaling, a standard scaler is performed.
5. The target variables are set.
6. Next, the dataset was separated into input features (X) and output labels (y) minus the target and the original Confirmed column.
7. Train-test split was performed.
8. Train a decision tree classifier on the training data so that the limit on depth (e.g., `max_depth=5`) is imposed to limit overfitting and recursion error.
9. A visual plot of the trained decision tree can be displayed using `plot_tree()` from scikit-learn.

Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.mixture import GaussianMixture

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.model_selection import train_test_split

from sklearn.metrics import classification_report, accuracy_score

# Load data

data = pd.read_csv('/content/covid_19_india.csv')

# Drop unnecessary columns

data.drop(['Sno', 'Date', 'Time'], axis=1, inplace=True)

# Label Encoding & conversion

le = LabelEncoder()

data['State/UnionTerritory'] = le.fit_transform(data['State/UnionTerritory'])

data['ConfirmedIndianNational'] = pd.to_numeric(data['ConfirmedIndianNational'],

errors='coerce').fillna(0).astype(int)

data['ConfirmedForeignNational'] = pd.to_numeric(data['ConfirmedForeignNational'],

errors='coerce').fillna(0).astype(int)

# Standardize the data

scaler = StandardScaler()

scaled_data = scaler.fit_transform(data)

# Step 1: Apply Gaussian Mixture Model

gmm = GaussianMixture(n_components=3, random_state=0)

gmm_labels = gmm.fit_predict(scaled_data)
```

```

# Add GMM cluster labels as new column

data['GMM_Cluster'] = gmm_labels

# Step 2: Create a label for classification (e.g., Confirmed cases categories)

# Categorize confirmed cases into 3 classes: Low, Medium, High

def case_category(x):

    if x < 100:

        return 0 # Low

    elif x < 1000:

        return 1 # Medium

    else:

        return 2 # High

data['CaseCategory'] = data['Confirmed'].apply(case_category)

# Prepare features and label

X = data.drop(['Confirmed', 'CaseCategory'], axis=1)

y = data['CaseCategory']

# Train-test split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Step 3: Decision Tree Classifier

dt_model = DecisionTreeClassifier(max_depth=5, random_state=42)

dt_model.fit(X_train, y_train)

# Predict and evaluate

y_pred = dt_model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

print("Classification Report:\n", classification_report(y_test, y_pred))

# Step 4: Visualize the decision tree

plt.figure(figsize=(16, 10))

```

```

plot_tree(dt_model, feature_names=X.columns, class_names=['Low', 'Medium', 'High'],
filled=True)

plt.title("Decision Tree Using GMM Cluster Feature")

plt.show()

```

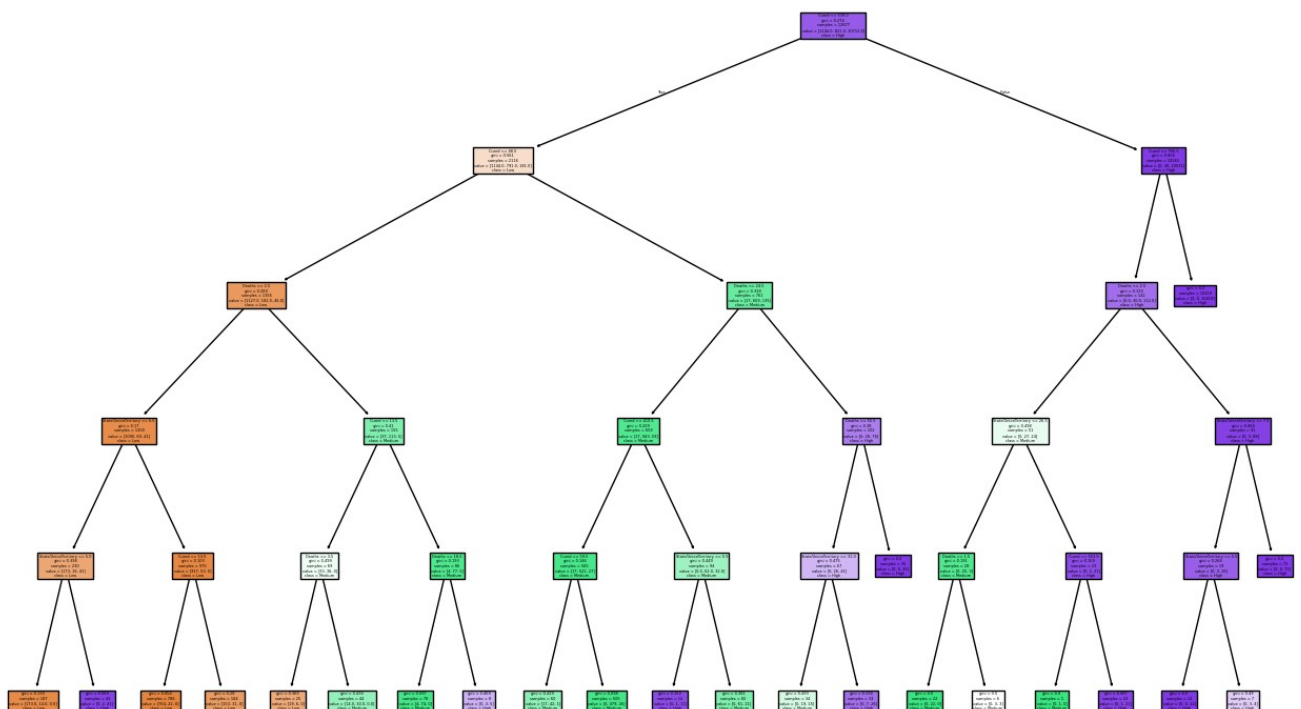
Output:

Accuracy: 0.9823302043070127

Classification Report:

	precision	recall	f1-score	support
0	0.93	0.99	0.96	498
1	0.87	0.85	0.86	345
2	1.00	0.99	0.99	4590
accuracy			0.98	5433
macro avg	0.93	0.94	0.94	5433
weighted avg	0.98	0.98	0.98	5433

Decision Tree Using GMM Cluster Feature



Result:

Thus the python program for Decision tree using Gaussian Mixture Model was executed successfully

EXPTNO :7 (b) **Implementation of Random Forest Tree.**

Construct python program for random forest tree and build the model

Aim:

ToConstruct python program for random forest tree and build the model .

Procedure:

1. Import the necessary libraries like `pandas`, `sklearn`, and `matplotlib`.
2. We use the `Iris` dataset here for simplicity. Replace it with your dataset (using `pd.read_csv()` for CSV files).
3. We separate the dataset into independent variables (features) and dependent variable (target). Split the data into training and testing sets (80-20 split in this case).
4. Initialize the `RandomForestClassifier` and train it with the training data.
5. Use the trained model to predict values for the test data.
6. We evaluate the model using accuracy score and a classification report.

Code:

```
# Step 1: Importing the necessary libraries
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# Step 2: Load a dataset (using Iris dataset as an example here)
# You can replace this with your own dataset
data = load_iris()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['target'] = data.target

# Step 3: Extracting Independent and Dependent Variables
X = df.drop(columns=['target']) # Independent variables (features)
y = df['target'] # Dependent variable (target)

# Step 4: Splitting the dataset into Training and Test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 5: Fitting the Random Forest Classifier to the Training Set
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Step 6: Predicting the Test Set Results
y_pred = model.predict(X_test)

# Step 7: Evaluating the Model
accuracy = accuracy_score(y_test, y_pred)
```

```

print(f'Accuracy: {accuracy * 100:.2f}%%')
print("Classification Report:")
print(classification_report(y_test, y_pred))

# Optional: Visualizing feature importance
plt.barh(X.columns, model.feature_importances_)
plt.xlabel('Feature Importance')
plt.ylabel('Feature')
plt.title('Random Forest Feature Importance')
plt.show()

```

Output:

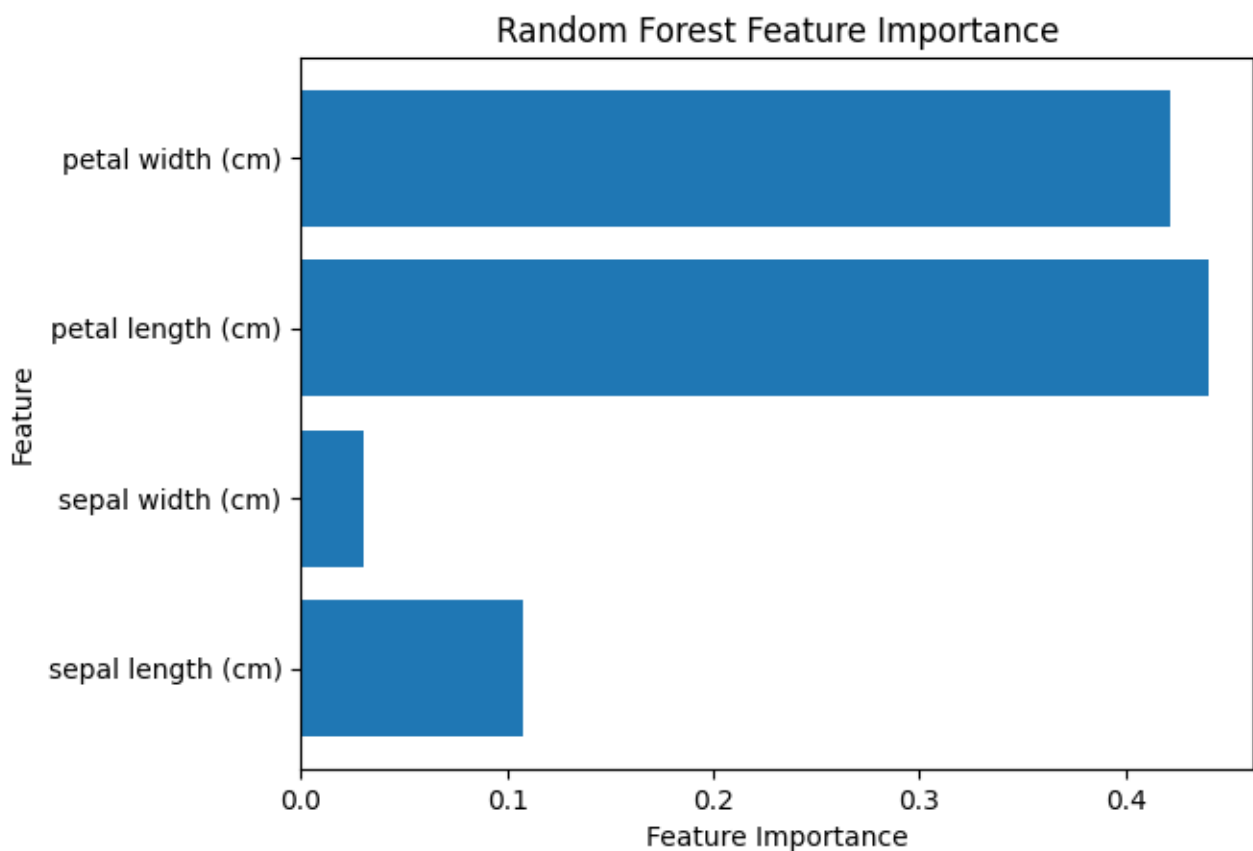
```

Accuracy: 100.00%
Classification Report:
precision    recall  f1-score   support

         0         1.00      1.00      1.00         10
         1         1.00      1.00      1.00          9
         2         1.00      1.00      1.00         11

accuracy
macroavg      1.00      1.00      1.00         30
weightedavg   1.00      1.00      1.00         30

```



Result

Thus, the python program for random forest tree was executed successfully

Ex. No. 3

Propositional Model Checking using Python

Aim

To implement a simple propositional model-checking algorithm using Python that verifies whether a given propositional formula is true under different truth assignments.

Algorithm for Propositional Model Checking

Input:

A Boolean expression with propositional variables (e.g., "(p or q) and (not p or q)").

Steps:

1. **Identify Variables:**
 - Extract all unique propositional variables (p, q, r, ...) from the given Boolean expression.
2. **Generate Truth Assignments:**
 - Create all possible combinations of True and False values for the extracted variables.
3. **Evaluate Expression:**
 - Substitute each truth assignment into the Boolean expression.
 - Compute the truth value using Python's eval() function.
4. **Output Truth Table:**
 - Display all possible truth assignments along with the corresponding evaluation results.
5. **Classify the Formula:**
 - If the formula is True for all truth assignments → **Tautology**.
 - If the formula is True for at least one truth assignment but not all → **Satisfiable but not a Tautology**.
 - If the formula is False for all truth assignments → **Contradiction**.
6. **Output the Final Result.**

Program:

```
from itertools import product

# Function to evaluate a Boolean expression under a given truth assignment
def evaluate_expression(expression, variables, values):
    env = dict(zip(variables, values)) # Map variables to their truth values
    return eval(expression, {}, env) # Evaluate the expression using the assigned values

# Function to perform propositional model checking
def propositional_model_check(expression):
    valid_vars = {'p', 'q', 'r'} # Define valid propositional variables
```

```

variables = sorted(set(c for c in expression if c in valid_vars)) # Extract unique variables
truth_assignments = list(product([False, True], repeat=len(variables))) # Generate truth table
assignments

results = [evaluate_expression(expression, variables, assignment) for assignment in
truth_assignments]

# Display the Truth Table
print("Truth Table:")
print(" | ".join(variables) + " | Result")
print("-" * (len(variables) * 4 + 9))
for assignment, result in zip(truth_assignments, results):
    print(" | ".join(str(int(val)) for val in assignment) + " | {int(result)}")

# Determine the classification of the Boolean formula
if all(results):
    return "The formula is a Tautology (always True)."
elif any(results):
    return "The formula is Satisfiable but not a Tautology."
else:
    return "The formula is a Contradiction (always False)."

# Example Usage
expression = "(p or q) and (not p or q)" # Example Boolean formula
result = propositional_model_check(expression)
print("\nResult:", result)

```

Output

Truth Table:

p | q | Result

0 | 0 | 0

0 | 1 | 1

1 | 0 | 1

1 | 1 | 1

Result: The formula is Satisfiable but not a Tautology.

Result:

Thus the program to implement Propositional Model checking Algorithm is implemented and executed successfully.

AIM:

To Construct and implement propositional model checking algorithms using python

Procedure:

1. Define a class Literal with attributes name and sign to denote whether the literal is positive or negative.
2. Implement the `__neg__` function to return a new literal with the same name but the opposite sign of its parent literal.
3. Implement the `__repr__` function to return the string of the literal name (or the string with a negative sign) each time the instance of the literal is called.
4. Create the `CNFConvert` function to convert the knowledge base (KB) from a list of sets to a list of lists for easier computing.
5. Create the `VariableSet` function to find all the used literals in the KB, assisting with running the DPLL algorithm.
6. Implement the `Negativeofx` function to hold the negative form of the literal for use in the DPLL algorithm.
7. Create the `pickX` function to pick a literal from the variable set and work with it as a node in the tree.
8. Define the functions `splitfalseliterals()` and `splitTrueLiteral()`.
9. Create the `dpll()` function to perform the DPLL algorithm recursively.
10. Finally, call the function to execute the code.

Code:

```
import re
```

```
class Literal:
```

```
    # Class Literal, it has attributes name and sign to denote whether the literal is positive or  
negative in use
```

```
    def __init__(self, name, sign=True):
```

```
        self.name = str(name)
```

```
        self.sign = sign
```

```
    def __neg__(self): # Returns a new literal with the same name but the opposite sign of its  
parent literal
```

```
        return Literal(self.name, False)
```

```
    def __str__(self):
```

```
        return str(self.name)
```

```
    def __repr__(self):
```

```
        # Returns the string of the literal name (or the string with a negative sign) each time the  
instance of the literal is called
```

```
        if self.sign:
```

```
            return '%r' % str(self.__str__())
```

```
        else:
```

```
            return '%r' % str("-" + self.__str__())
```

```
def CNFconvert(KB):

    # This function converts the KB from a list of sets to a list of lists for easier computing

    storage = []

    for i in KB:

        i = list(i)

        for j in i:

            j = str(j)

        storage.append(i)

    return storage
```

```
def VariableSet(KB):

    # This function finds all the used literals in the KB, and in order to assist with running the
    DPLL
```

```
    KB = eval((CNFconvert(KB).__str__()))

    storage = []

    for obj in KB:

        for item in obj:

            if item[0] == '-' and item[1:] not in storage:

                storage.append(str(item[1:]))

            elif item not in storage and item[0] != '-':

                storage.append(str(item))

    return storage
```

```
def Negativeofx(x):

    # This function is for holding the negative form of the literal, for use in the DPLL algorithm

    check = re.match("-", str(x))

    if check:
```

```
    return str(x[1:])
```

```
else:
```

```
    return "-" + str(x)
```

```
def pickX(literals, varList):
```

```
    # This function picks a literal from the variable set and works with it as a node in the tree
```

```
    for x in varList:
```

```
        if x not in literals:
```

```
            break
```

```
    return x
```

```
def splitFalseLiterals(cnf, x):
```

```
    holder = []
```

```
    for item in cnf:
```

```
        if x in item:
```

```
            item.remove(x)
```

```
            holder.append(item)
```

```
    return holder
```

```
def splitTrueLiteral(cnf, x):
```

```
    holder = []
```

```
    for item in cnf:
```

```
        if x in item:
```

```
            continue
```

```
        else:
```

```
            holder.append(item)
```

```
    return holder
```

```

def unitResolution(clauses):

    literalholder = {} # Dictionary for holding the literal holder and their bool

    i = 0

    # This part of the code goes through each and every clause until all literals in the KB are
    resolved

    while i < len(clauses): # For each clause

        newClauses = []

        clause = clauses[i]

        # Picks a clause to work on

        if len(clause) == 1:

            literal = str(clause[0])

            pattern = re.match("-", literal)

            # Populates the dictionary

            if pattern:

                nx = literal[1:]

                literalholder[nx] = False

            else:

                nx = "-" + literal

                literalholder[literal] = True

            # Checks for all other appearances of the literal or its opposite in the KB

            for item in clauses:

                if item != clause[i]:

                    if nx in item:

                        item.remove(nx)

                    newClauses.append(item)

            i = 0

```

```

        clauses = newClauses

    # No unit clause

    else:

        i += 1

    return literalholder, clauses


def dpll(clauses, varList):

    # Recursively performs the DPLL algorithm

    literals, cnf = unitResolution(clauses)

    if cnf == []:

        return literals

    elif [] in cnf:

        return "notsatisfiable"

    else:

        # Pick a literal which isn't set yet but has an impact on the KB, and then work on it
        recursively

        while True:

            x = pickX(literals, varList)

            x = str(x)

            nx = Negativeofx(x)

            ncnf = splitTrueLiteral(cnf, x)

            ncnf = splitFalseLiterals(ncnf, nx)

            if ncnf == cnf:

                varList.remove(x)

            else:

                break

        # Does the same DPLL recursively, but follows the true path for that variable

```

```
case1 = dpll(ncnf, varList)
```

```
if case1 != "notsatisfiable":
```

```
    copy = case1.copy()
```

```
    copy.update(literals)
```

```
    copy.update({x: True})
```

```
    return copy
```

```
# Does the DPLL recursively, but follows the false path for that variable
```

```
case1 = dpll(ncnf, varList)
```

```
if case1:
```

```
    copy = case1.copy()
```

```
    copy.update(literals)
```

```
    copy.update({x: False})
```

```
    return copy
```

```
else:
```

```
    return "notsatisfiable"
```

```
def DPLL(KB):
```

```
    # Finally restructures the output to fit the required output by the assignment description
```

```
    KB = eval((CNFconvert(KB).__str__()))
```

```
    varList = VariableSet(KB)
```

```
    result = dpll(KB, varList)
```

```
    if result == 'notsatisfiable':
```

```
        return False
```

```
    else:
```

```
        for i in varList:
```

```
            if i in result and result[i] == True:
```

```
                result[i] = 'true'
```

```
elif i in result and result[i] == False:

    result[i] = 'false'

else:

    result[i] = 'free'

return [True, result]
```

```
A = Literal('A')
B = Literal('B')
C = Literal('C')
D = Literal('D')
KB = [{A, B}, {A, -C}, {-A, B, D}]
print(DPLL(KB))
```

OUTPUT:

```
[True, {'A': 'true', 'B': True, 'C': 'free', 'D': 'free'}]
```

Result:

Thus the program to implement Propositional Model checking Algorithm is implemented and executed successfully.

